

**MAŁOPOLSKA UCZELNIA PAŃSTWOWA
IMIENIA ROTMISTRZA WITOLDA PILECKIEGO
W OŚWIĘCIMIU**

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Rok studiów/semestr: rok III, semestr VI

Patryk Wlik

nr albumu 8793

**PROJEKT APLIKACJI MOBILNEJ ZINTEGROWANEJ
Z BAZĄ DANYCH**

Projekt na przedmiot systemy mobilne

Prowadzący: mgr inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imię i nazwisko autora projektu	Patryk Wlik
Rok urodzenia autora projektu	2003
Stopień/tytuł zawodowy, imię i nazwisko prowadzącego przedmiotu	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 29.04.2026 do 10.06.2026
Nazwa przedmiotu	systemy mobilne
Rok studiów/semestr	rok III, semestr VI

Tytuł projektu	Projekt aplikacji mobilnej zintegrowanej z bazą danych
Słowa kluczowe (max 5)	informatyka, aplikacje mobilne, bazy danych
Streszczenie (max 1400 znaków)	Aplikacja umożliwiająca wyświetlanie rekordy znajdujących się w bazie danych. Posiada interfejs użytkownika, który umożliwia pokazanie zapisanych w bazie danych. Takich jak godzina, data czy temperatura.

Spis treści

Wstęp	4
1. Cel i zakres pracy	5
1.1. Cel pracy	5
1.2. Cele szczegółowe	5
1.3. Zakres pracy	5
2. Analiza wymagań i projekt rozwiązania	6
2.1. Wymagania funkcjonalne	6
2.2. Wymagania нефункционалне	6
2.3. Architektura rozwiązania	6
2.4. Warstwa sprzętowa	6
2.5. Warstwa serwera	6
2.6. Warstwa bazy danych	7
2.7. Warstwa aplikacji mobilnej	7
3. Opis działania aplikacji	8
3.1. Środowisko i narzędzia	8
3.2. Struktura projektu	8
3.3. Interfejs użytkownika	8
3.4. Pobieranie danych z REST API	9
3.5. Automatyczne odświeżanie co 10 sekund	9
3.6. Logika wyświetlania aktualnej temperatury	9
3.7. Wyświetlanie dziennika	9
3.8. Diagram klas	10
3.9. Diagram sekwencji	10
4. Implementacja i uruchomienie	11
4.1. Wymagania sprzętowe	11
4.2. Instrukcja instalacji	11
4.3. Instrukcja użytkowania	11
5. Testy i weryfikacja	12
5.1. Testy funkcjonalne	12
5.2. Testy нефункционалне	12
5.3. Wyniki testów	12
Podsumowanie	13
Spis rysunków	14
Spis tabel	15
Spis listingów	17

Wstęp

Rozwój Internetu Rzeczy (IoT) pozwala na zdalne śledzenie warunków otoczenia w czasie rzeczywistym. Kluczowym wskaźnikiem, monitorowanym zarówno w domach, jak i w przemyśle, jest temperatura. Projekt ten opisuje budowę systemu opartego na mikrokontrolerze ESP32 WROOM z czujnikiem DHT11, który współpracuje z bazą danych MySQL oraz dedykowaną aplikacją na system Android. Rozwiązanie to pozwala nie tylko gromadzić pomiary, ale również wygodnie przeglądać aktualne odczyty i historię danych bezpośrednio na smartfonie.

1. Cel i zakres pracy

1.1. Cel pracy

Celem pracy jest zaprojektowanie i implementacja aplikacji mobilnej w środowisku Android Studio, zintegrowanej z bazą danych MySQL, umożliwiającej odczyt i prezentację danych temperatury pochodzących z czujnika DHT11 podłączonego do modułu ESP32 WROOM.

1.2. Cele szczegółowe

W ramach realizacji projektu wyznaczono następujące cele szczegółowe:

- opracowanie interfejsu API do komunikacji aplikacji z bazą danych,
- zaprojektowanie aplikacji mobilnej w Android Studio,
- implementacja mechanizmu pobierania danych z bazy (REST API, JSON),
- wyświetlanie aktualnej temperatury oraz dziennika pomiarów w aplikacji,
- przeprowadzenie testów funkcjonalnych i нефункциональных.

1.3. Zakres pracy

Zakres pracy obejmuje:

- konfigurację środowiska MySQL,
- implementację RestAPI,
- stworzenie aplikacji mobilnej Android,
- testowanie systemu.

2. Analiza wymagań i projekt rozwiązania

Rozdział przedstawia wymagania oraz projekt rozwiązania. Opis obejmuje zarówno aspekty funkcjonalne, jak i нефункционалне, a także opis architektury.

2.1. Wymagania funkcjonalne

Do kluczowych wymagań funkcjonalnych należą:

- aplikacja musi łączyć się z serwerem opartym o SpingBoot
- zapis danych do lokalnej relacyjnej bazy SQL lub nierelacyjnej na serwerze
- aplikacja musi pobierać aktualny pomiar temperatury,
- aplikacja musi wyświetlać dziennik zapisów,
- dane muszą być prezentowane w sposób czytelny,
- integracja z urządzeniami RestAPI,
- przygotowanie dokumentacji zgodnej z wymaganiami edytorskimi.

2.2. Wymagania нефункционалне

Rozwiązanie powinno spełniać następujące wymagania jakościowe:

- kompatybilność z systemami Android 8.0+,
- czas odpowiedzi API mniejszy niż 300ms,
- dostępność systemu większa niż 99 procent,
- szyfrowanie SSL/TLS dla transmisji danych,
- dostępność zgodna z WCAG 2.1 na poziomie AA.

2.3. Architektura rozwiązania

System opiera się na architekturze warstwowej oraz modelu klient–serwer, co pozwala na wyraźny podział zadań między poszczególne komponenty. Takie podejście sprawia, że projekt jest przejrzysty i łatwy w rozbudowie, a zmiany w jednym elemencie nie zakłócają pracy pozostałych. Całość struktury tworzą cztery główne filary: warstwa sprzętowa, warstwa serwera, warstwa baza danych oraz warstwa aplikacji mobilnej.

2.4. Warstwa sprzętowa

- Serwer,
- Telefon,

2.5. Warstwa serwera

- MySQL,
- SpringBoot + RestAPI,

API odpowiada za:

- udostępnianie danych aplikacji mobilnej w formacie JSON.

2.6. Warstwa bazy danych

Warstwa danych została stworzona aby umożliwić zapisywanie wymaganych danych tak jak pokazano w tabeli 1. Taka struktura danych umożliwia spełnienie założonych wymagań.

Tabela 1
Struktura tabeli temperatura

Pole	Typ danych
id	int
data	date
godzina	int
temperatura	int

Źródło: opracowanie własne

2.7. Warstwa aplikacji mobilnej

Aplikacja Android:

- komunikacja z API,
- przetwarzanie JSON,
- wyświetlanie danych w UI.

3. Opis działania aplikacji

Rozdział opisuje logikę funkcjonowania aplikacji poprzez opisanie jej poszczególnych funkcjonalności oraz przedstawienie kodu.

3.1. Środowisko i narzędzia

- Android Studio,
- Język Java,
- MySQL,
- Serwer SpringBoot,
- Docker.

3.2. Struktura projektu

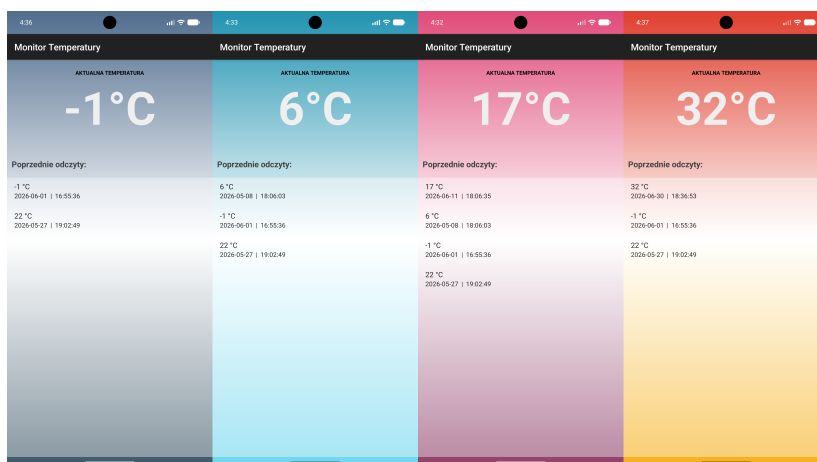
Struktura projektu składa się z plików projektu aplikacji Android Studio, projektu aplikacji serwera SpringBoot w IntelliJ, oraz bazy danych implementowanej za pomocą Docker Desktop.

3.3. Interfejs użytkownika

Na górze interfejsu umieszczony jest napis "aktualna temperatura" a pod nim wyświetlane są najnowsze dane temperatury z bazy.

Poniżej znajduje się dynamiczna lista która wyświetla temperaturę, godzinę i datę poprzednich pomiarów.

Interfejs został zaprojektowany aby być czytelny i prosty w użytkowaniu. Zmienia on wygląd w zależności od aktualnej temperatury, tak jak na rysunku 1. Dzięki temu interfejs wygląda bardziej nowocześnie i unikatowo.



Rysunek 1. Interface aplikacji

Źródło: opracowanie własne przy użyciu emulatora Android Studio

3.4. Pobieranie danych z REST API

Aplikacja pobiera dane asynchronicznie za pomocą Retrofita, co zapobiega blokowaniu interfejsu i zapewnia wysoką responsywność.

Listing 1. Pobieranie danych z API (Fragment ApiService.java)

```
1 @GET("api/pomiary")
2 Call<List<Dane>> getWszystkiePomiary();
```

3.5. Automatyczne odświeżanie co 10 sekund

Wykorzystano mechanizm Handler i Runnable do stworzenia pętli czasowej, która działa w sposób inteligentny: aktywuje się, gdy korzystasz z aplikacji, i automatycznie pauzuje po jej zminimalizowaniu.

Listing 2. Odświeżanie (Fragment MainActivity.java)

```
1 refreshRunnable = new Runnable() {
2     @Override
3     public void run() {
4         pobierzDaneZSerwera(); // Wywołanie API
5         handler.postDelayed(this, 10000); // Ponowne uruchomienie za 10s
6     }
7 };
```

3.6. Logika wyświetlania aktualnej temperatury

Zamiast pobierać temperaturę pojedynczo, aplikacja ładuje całą listę i wskazuje jej ostatni element jako bieżący stan. Dzięki temu użytkownik zawsze widzi najświeższy odczyt bez zbędnych zapytań.

Listing 3. Wyświetlanie "Aktualnej temperatury" (Fragment MainActivity.java)

```
1 if (!daneList.isEmpty()) {
2     Dane ostatni = daneList.get(daneList.size() - 1); // Pobranie najnowszego
    wpisu
3     tvCurrentTemp.setText(ostatni.getTemperatura() + "°C");
4     Collections.reverse(daneList); // Odwrocenie listy dla historii (najnowsze
    na gorze)
5 }
```

3.7. Wyświetlanie dziennika

Adapter przetwarza pobrane dane i automatycznie przypisuje je do właściwych pól tekstowych w widoku aplikacji.

Listing 4. Wyświetlanie dziennika (Fragment DaneAdapter.java)

```
1 @Override
2 public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
3     Dane d = lista.get(position);
```

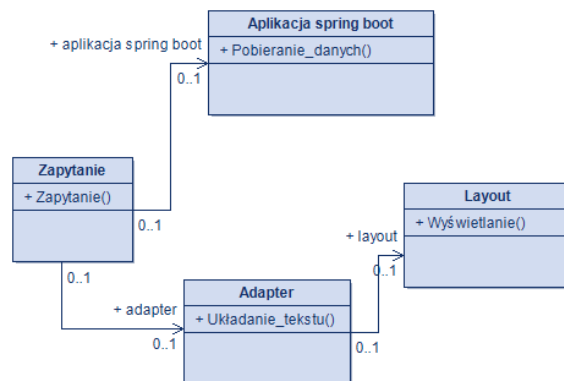
```

4 holder.text1.setText(d.getTemperatura() + "'C");
5 holder.text2.setText(d.getData() + "||" + d.getGodzina());
6 }

```

3.8. Diagram klas

Diagram pokazuje jak podzielona jest aplikacja oraz idea jej działania.

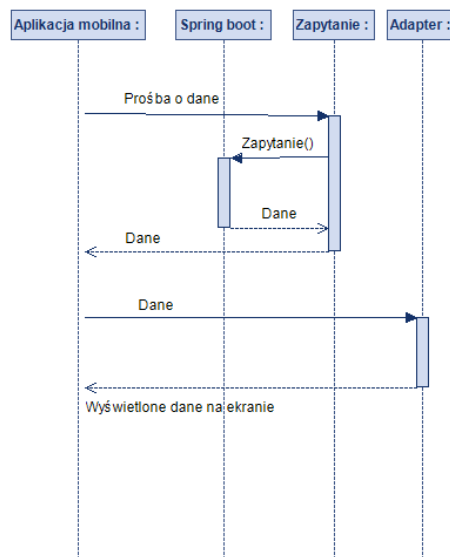


Rysunek 2. Diagram klas

Źródło: opracowanie własne przy użyciu modelio

3.9. Diagram sekwencji

Diagram pokazuje jak w aplikacji przesyłane są dane.



Rysunek 3. Diagram sekwencji

Źródło: opracowanie własne przy użyciu modelio

4. Implementacja i uruchomienie

Instrukcja zawiera informacje potrzebne do instalacji i użytkowania aplikacji.

4.1. Wymagania sprzętowe

Po stronie użytkownika:

- Smartfon z Android 8.0+
- Połączenie Wi-Fi

Po stronie serwera:

- Serwer z MySQL
- Serwer z aplikacją SpringBoot

4.2. Instrukcja instalacji

- Połącz komputer (serwer) i telefon do tej samej sieci Wi-Fi.
- Zainstaluj i włącz plik .apk
- Uruchom bazę danych Docker/MySQL oraz aplikację SpringBoot.
- Włącz aplikację

4.3. Instrukcja użytkowania

Aplikacja od razu po starcie prezentuje bieżącą temperaturę oraz pełną historię pomiarów. Dzięki automatycznej aktualizacji co 10 sekund użytkownik ma stały wgląd w najnowsze dane bez żadnego wysiłku. Co ważne, proces ten jest w pełni zoptymalizowany – system pobiera dane tylko wtedy, gdy aplikacja jest widoczna na ekranie, co pozwala oszczędzać energię urządzenia.

5. Testy i weryfikacja

Rozdział przedstawia sposób sprawdzenia poprawności działania aplikacji oraz ocenę uzyskanych rezultatów. Testy skupiają się na weryfikacji poprawności działania oraz zgodności z wymaganiami funkcjonalnymi i нефункциональными.

5.1. Testy funkcjonalne

Tabela 2

Tabela wyników testów funkcjonalnych

Test	Oczekiwany Wyniki	Wynik
Połączenie z API	Dane zwracane w JSON	Pozytywny
Wyświetlanie aktualnej temp.	Widoczna poprawna wartość	Pozytywny
Wyświetlanie dziennika	Lista rekordów	Pozytywny

Źródło: opracowanie własne

5.2. Testy нефункционаalne

Test wydajności

- Czas odpowiedzi API: akceptowalny
- Aplikacja reaguje płynnie.

Test kompatybilności

- Android 8, 10, 13 – działanie poprawne.

5.3. Wyniki testów

System pracuje stabilnie, zapewniając bezbłędną wymianę danych między układem ESP32 a aplikacją mobilną. Procesy zapisu i odczytu danych z bazy przebiegają prawidłowo, a intuicyjny design sprawia, że interfejs jest przejrzysty i przyjazny w codziennym użytkowaniu.

Podsumowanie

Projekt umożliwia wyświetlenie danych pobranych z bazy przez serwer SpringBoot na telefonie. Dzięki czemu ma możliwość szczytywania danych np. z czujnika temperatury, co pozwala monitorować aktualne parametry środowiska. Po niewielkich zmianach może służyć jako szablon do podobnych aplikacji.

Projekt może zostać w przyszłości rozszerzony o:

- wykresy temperatur,
- powiadomienia o przekroczeniu progów,
- bardziej przyjemny wizualnie,

Spis rysunków

<i>Rysunek 1.</i> Interface aplikacji	8
<i>Rysunek 2.</i> Diagram klas	10
<i>Rysunek 3.</i> Diagram sekwencji	10

Spis tabel

Tabela 1. Struktura tabeli temperatura	7
Tabela 2. Tabela wyników testów funkcjonalnych	12

Spis listingów

Listing 1. Pobieranie danych z API (Fragment ApiService.java)	9
Listing 2. Odświeżanie (Fragment MainActivity.java)	9
Listing 3. Wyświetlanie "Aktualnej temperatury"(Fragment MainActivity.java) . . .	9
Listing 4. Wyświetlanie diennika (Fragment DaneAdapter.java)	9